Team 29: Siri Adsumilli and Juhi Parikh

Design Description of System:

For this project, the first step we undertook was to understand the flow of information and how the ATM, Router, and Bank are connected. We worked to implement a banking system that communicated over UDP on localhost. The ATM client sends commands to the Bank server through the Router proxy and the Bank stores account information while also having the ability to create accounts. This architecture between the ATM, router, and bank makes it so the bank has both local and remote commands which are dealt with differently in the code. The ATM had a local session based on the authenticated user and changed information in that account with commands sent through the router to the bank. The router was the proxy connection between the Bank and the ATM. These three main branches of the program work together to create this virtual banking system.

First, the bank program acted as the server and listened for messages from the ATM. We created a hash table with 100 different possible accounts all of which had a username, 4-digit pin, and balance. The bank had the ability to create a user, deposit money, and give you the current balance. It also worked to answer remote operations from the ATM that used the accounts in the bank. We followed all of the specific restrictions given about the kind of information that can be inputted to create an account and generated a .card file each time an account was created. In these files, we made sure to emphasize the importance of formatting checks in the username, pin, and balance when we created a user to protect from pin enumeration attacks, format attacks and injection prevention. To prevent buffer overflows in this virtual bank, we also made sure to have a bounded string copy of the command parsed which was a 1024 byte buffer.

Second, the ATM program acts as a function of the client sending requests through the router to the bank. It also had a local active session based on the user that is authenticated and can communicate changes in the account to the bank. We used extra helper methods to check if the .card files exist to access the accounts and this helps make sure we have stronger authentication capabilities. Our use of a variable to check if a session is active in this section also makes sure that we can implement proper session cleanup on the user logout while also resetting the active flag. This protects from unauthorized user execution without an active session. We make sure that each username is bound to a specific session so that a user cannot access another user's credentials in their own session. This protects from authorization bypass. In the same case as the bank, to prevent buffer overflows, we made sure to have a bounded string copy of the command parsed which was a 1024 byte buffer and check for correct formatting of the username to prevent injection.

The router was given code that makes up the UDP connection to create this proxy connection between the ATM and Bank. We created it to accept a filename as an argument to check whether the file already exists before creation so we can prevent accidental overwrites of existing files. Both files we used are simply placeholders, but are here to store cryptographic

keys for this secure communication between the ATM and the Bank. There are specific exit codes for incorrect file handling which each tell us something different based on the number returned on exit.

This project gave us the opportunity to learn how to start from scratch in a way to implement a distributed networking system with many different parts. We learned about defense-in-depth which we went over in class and worked to create this defense to protect from unauthorized command execution, enumeration attacks, buffer overflow, authentication bypass, and injections. We created a system that isn't at complete capacity in how secure it is and learned a lot from looking at different perspectives to see how potential threats could arise in our code. We were able to sharpen our skills in C which we haven't used in a while and were able to follow through on a project with many different specifications in different architectural spaces.

Vulnerabilities:

1. Name of Vulnerability: Unauthorized Command Execution Without Active Sessions
    a. Mitigation Described: To protect against unauthorized command execution, we incorporate session state validation. Without a valid active session, attackers are not able to withdraw money due to a lack of valid credentials. We have set up a flag to make sure that it checks if the credentials are valid to start an active session in order to withdraw safely. If the flag is 0 (false) then it is set to have no user authentication logged in and nothing is sent to the bank. This makes sure that the gold standard is in place with authentication, authorization, and audibility checks. Authentication is in place because users have to make sure all credentials are correct to be able to start a session. Authorization is in place because our program makes sure that users have an active session to be able to grant permissions to any other operations that involve the bank and the user's account. Audibility is in place because we are able to reject any faulty credential login attempts and not allow that to result in an active session. Whether or not it's a user or an attacker, there is security preserved in that process.
    b. Mitigation Implemented:
        i.

```
if (!atm->active)
        {
            printf("No user logged in\n");
            return;
        }
        if (block_count != 2)
        {
            printf("Usage: withdraw <amount>\n");
```

```
            return;
        }
if (!atm->active)
        {
            printf("No user logged in\n");
            return;
        }

    if (!atm->active)
        {
            printf("No user logged in\n");
            return;
        }
```

ii. Description of Code: These are all examples of checks we created to make sure that an active session is in place to proceed to any other activities. In all of these examples provided above, the inactive sessions cause a print statement of "no user logged in" and comes out of the check without proceeding. It terminates the ability to go further without having an active session.

2. Name of Vulnerability: Pin Enumeration Attack Via Strong Authentication
   a. Mitigation Described: To protect against enumeration attacks, we incorporate authentication and auditability checks. With the possibility of enumeration attacks, our implementation is secure because it makes sure that the PIN is unique enough for attackers to not be able to guess as easily with a weak password. The pin is 4 digits and with 10 possible numerical choices for each digit, that results in 10,000 possible choices. That is a very large range of possible passwords, which makes it difficult for attackers to easily identify. Because users are strictly enforced to input numbers in the pins, they are not able to input extra characters which upholds the audibility process. Due to this, the likelihood of our implementation being compromised is unlikely and makes sure that it follows gold standard properties of a secure system.
   b. Mitigation Implemented:

```
int pin = atoi(pin_str);


        char *endptr;
        long balance_long = strtol(balance_str, &endptr, 10);
        if (*endptr != '\0' || balance_long < 0 || balance_long >
INT_MAX)
        {
            printf("Usage:  create-user <user-name> <pin>
<balance>\n");
```

```
        return;
    }
    int balance = (int)balance_long;

    if (hash_table_find(bank->accounts, username) != NULL)
    {
        printf("Error: user %s already exists\n", username);
        return;
    }
```

i. Description of code: Our code makes sure the length is 4 digits and is not easily brute force-able. We also make sure they are all numerical digits so special characters are not included for possible injection attacks through parsing issues.

3. Name of Vulnerability: Buffer Overflow - Unbounded String Copy
   a. Mitigation Described: To protect against Buffer Overflows, we implemented unbounded string copy with explicit null termination. We experience buffer overflows when data is exceeding the allocated buffer size which leads to overwriting information in memory. An attacker can exploit a buffer by overflowing it with lengthy command strings that eventually overwrites the stack and grants access to program execution. Now we are able to mitigate this issue by using strncpy() which helps limit the size of command_parsed to make sure that no more than 1024 bytes are being copied. We also make sure that the string is null-terminated to prevent any issues related to strings that are not terminated. Our implementation makes sure it's safe from potential buffer overflows and follows the gold standard principles by maintaining authentication and authorization. Hackers are not able to hijack the control flow of the login mechanisms which preserves the authentication process. Our actions are logged and verified which ensures that there aren't any vulnerabilities in memory which ensures authorization.
   b. Mitigation Implemented:
      i. In atm.c:

```
   char command_parsed[1024];

      strncpy(command_parsed, command,

   sizeof(command_parsed));

      command_parsed[sizeof(command_parsed)-1] = '\0';
4. ----- other code goes in between here -------
   strncpy(atm->user_logged_in, blocks[1],
   sizeof(atm->user_logged_in)-1);
```

      ii. In bank.c:

```
   strncpy(command_parsed, command, sizeof(command_parsed));
```

iii. Description of Code: In ATM, we are able to implement a fixed size buffer that only gets copied at that fixed length as well. We also implemented a null termination to make sure no other issues occur later on in the process. We also included an extra line to show that we ensure the user login does not overflow and has null termination. In bank.c we include the line to show that the attacker is not able to overwrite addresses with our bounded copy technique.

4. Name of Vulnerability: Authorization Bypass Through Access Prevention
   a. Mitigation Described: To prevent authorization bypass, we implemented automatic username binding to the authenticated session. If a hypothetical attacker is logged in as a user, in this case we can say Bob, then they could manipulate the system and gain access to other users accounts. This means they have access to withdraw money from those other accounts which shouldn't be allowed. We are able to work around this by ensuring we have a stored username for all operations done in an active session and they only have access to their account and .card file. We use the credentials from the login to get to the active session by searching to find the corresponding .card file. This means the user only has access to their account in their active session. This protects the gold standard by maintaining authentication, authorization, and audibility. Authentication is preserved because the username is stored during the login process. Authorization is preserved because users are only able to modify their own account and not have access to manipulate any other user's accounts. Audibility is preserved because the process is only valid if the login credentials are valid. So an active session cannot occur without credentials of a real account.
   b. Mitigation Implemented:
      i. Code:

```
atm->active = 1;

            strncpy(atm->user_logged_in, blocks[1],
sizeof(atm->user_logged_in)-1);
------------- other code in between ------------
char withdraw_message[256];
        snprintf(withdraw_message, sizeof(withdraw_message),
"withdraw %s %s", atm->user_logged_in, blocks[1]);
------------- other code in between -------------
char balance_message[256];
        snprintf(balance_message, sizeof(balance_message),
"balance %s", atm->user_logged_in);
----------- other code in between -------------
char end_message[256];
```

```
11.        snprintf(end_message, sizeof(end_message),
   "end-session %s", atm->user_logged_in);
```

ii. Description of Code: After we check if there is an active session in place, then we store the username. When withdrawing, there isn't even an option to specify which account to withdraw from, only monetary value is asked for; it's implied which account will be needed for doing this operation. For balance and end session operations it's the same logic, the account cannot be changed or specified by the end user.

5. Name of Vulnerability: Command Injection Via Special Character Exploitation
   a. Mitigation Described: To protect against command injection and special character attacks, we implemented strict input validation for usernames during account creation. Similar to the idea of SQL injection, where malicious code is inserted into a script, an attacker can insert malicious characters into a username to eventually exploit. We are able to mitigate this by utilizing isalpha() to validate usernames that only contain the alphabet and are up to 250 characters in length. This approach is cleaner and simpler than the approach of having a rulebook for characters that are not allowed. If we did the latter, attackers may still be able to find ways around blacklisted characters by finding other alternatives. We are upholding the gold standard by incorporating authentication, authorization, and audibility. Authentication is preserved by making sure the usernames are correctly built and correctly used throughout the application. Authorization is preserved by making sure that there are no path traversal attacks. This means that users are only able to do operations within their own account or "path", and not able to access any other accounts. Audibility is preserved because the usernames are created only by using alphabet characters and cannot have special characters which can lead to special character attacks and vulnerabilities that make the user name unreliable.
   b. Mitigation Implemented:
      i. Code:

```
for (int i = 0; username[i]; i++)
        {
            if (!isalpha(username[i]) || i >= 250)
            {
                printf("Usage:  create-user <user-name>
<pin> <balance>\n");
                return;
            }
        }
```

ii. Description of Code: The code is showing the use of isalpha() in the way we described. It iterates through the entire username to make sure everything is used from the alphabet and is less than 250 characters. This makes sure that there are no faulty usernames with special characters or other vulnerabilities.